



PRODI FISIKA
UNIVERSITAS BRAWIJAYA

Diseminasi Proyek Pemrograman Paralel 2: Kasus Deteksi Partikel Alpha Peluruhan Uranium-238

AHMAD HANAFI HASAN | 245090300111001
STEPHEN TANGGUH LAIA | 245090300111010

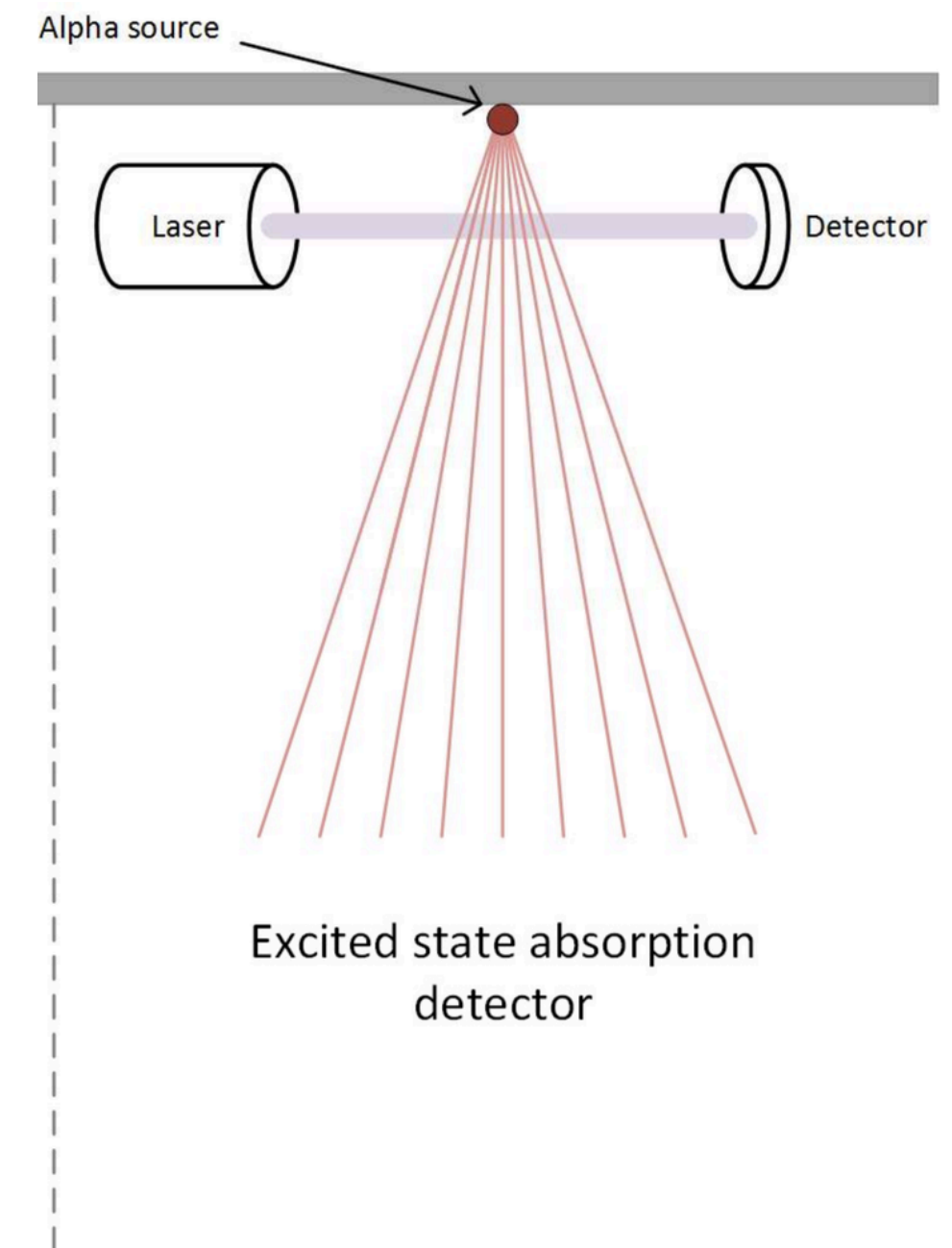
Latar Belakang

- Peluruhan secara nyata terlalu berbahaya untuk diamati secara langsung tanpa perlindungan radioaktif dan perlu waktu yang lama.
- Peluruhan U-238 dapat diamati seberapa cepat peluruhan melalui banyaknya partikel alpha yang dikeluarkan.
- Deteksi Partikel alpha dengan detektor nyata diperlukan biaya besar, dan dapat dilakukan secara simulasi.
- Simulasi detektor partikel alpha U-238 menghasilkan bentuk jejak partikel alpha yang dilihat detektor, dan akan perlu waktu lama untuk menghitung interaksinya jika dilakukan satu persatu.
- Diperlukan adanya bentuk paralelisme untuk mempercepat perhitungan dan visualisasi partikel tersebut.

Model Fisika

- Peluruhan = proses acak, mengikuti distribusi probabilitas yang eksponensial. setiap kejadian independen dan cocok untuk paralelisasi
- partikel alpha yang random dan saling tolak menolak karena muatan $+2e$ dan jumlahnya banyak, sesama alpha akan saling tolak menolak. Interaksi setiap partikel terhadap partikel lainnya membutuhkan komputasi berat sebesar $O(N)^2$
Diperlukan paralelisasi untuk menghitung interaksi antar partikel.
- Deteksi partikel alpha, berbentuk geometri kerucut mengacu pada peluruhan alpha yang arah gerakanya random dan diperlukan bentuk Monte Carlo untuk membuat kerandoman tersebut.

$$dt = -\frac{\ln(U)}{\lambda}$$

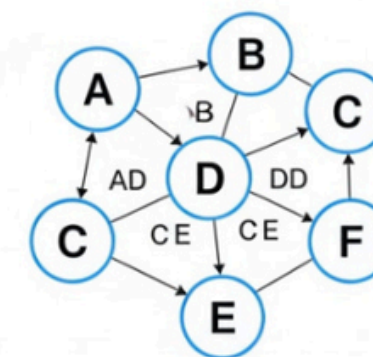


Strategi Paralelisasi

- Didasarkan pada karakteristik fisik dari eksperimen Monte Carlo.
- Menggunakan strategi Deomposisi Domain Data (Data Parallelism) dengan pendekatan “Embarrassingly Parallel”.
- Pemisahan beban kerja merata ke seluruh core, dengan 2 bentuk pendekatan : Shared Memory Architecture (lokal) dan Distributed Memory Architecture (HPC).
- Tiap core akan melakukan sampling Monte Carlo secara mandiri dan dikumpulkan kembali di akhir ke satu proses utama untuk divisualisasikan.



Serial



Each task depends on another.
Must be solved in sequence.

Embarrassingly parallel



No task depends on any other.
All solved simultaneously."

Implementasi

- Memakai bahasa Python dengan pendekatan hibrida (2 bentuk paralelisasi), tergantung mode yang dipakai.
- Lokal (-- mode MP), batasan GIL Python dihindari dengan metode multiprocessing

```
# — Mode Multiprocessing (core bisa diatur) —————  
def run_multiprocessing(  
    n: int,  
    workers: int | None,  
    seed: int | None,  
) -> tuple[ParticleBatch, int]:
```

```
# — Spawn Pool dengan k proses —————  
# _MP_CONTEXT = "fork" di Linux/Mac, "spawn" di Windows  
with get_context(_MP_CONTEXT).Pool(processes=k) as pool:  
    batches = pool.map(simulate_chunk, jobs)  
  
return combine_batches(batches), k
```

- get_context mengatur metode spawn atau fork (mengeksekusi batch)
- Beban kerja dibagi secara dinamis ke core yang ditentukan, melakukan Monte Carlo secara bersamaan, dan mengembalikan objek di fungsi ParticleBatch.

Implementasi

- Memakai bahasa Python dengan pendekatan hibrida (2 bentuk paralelisasi), tergantung mode yang dipakai.
- HPC (– – mode MPI)

```
# — Mode MPI —
def run_mpi(n: int, seed: int | None) -> tuple[ParticleBatch | None, int, int]:
    try:
        from mpi4py import MPI
    except ImportError as exc:
        raise SystemExit(
            "Mode MPI membutuhkan mpi4py. "
            "Install di environment HPC, lalu jalankan dengan mpiexec/mpirun/srun."
        ) from exc

    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    counts = split_counts(n, size) if rank == 0 else None
    seeds = child_seeds(seed, size) if rank == 0 else None

    local_count = comm.scatter(counts, root=0)
    local_seed = comm.scatter(seeds, root=0)
    local_batch = simulate_chunk((local_count, local_seed))
    gathered = comm.gather(local_batch, root=0)

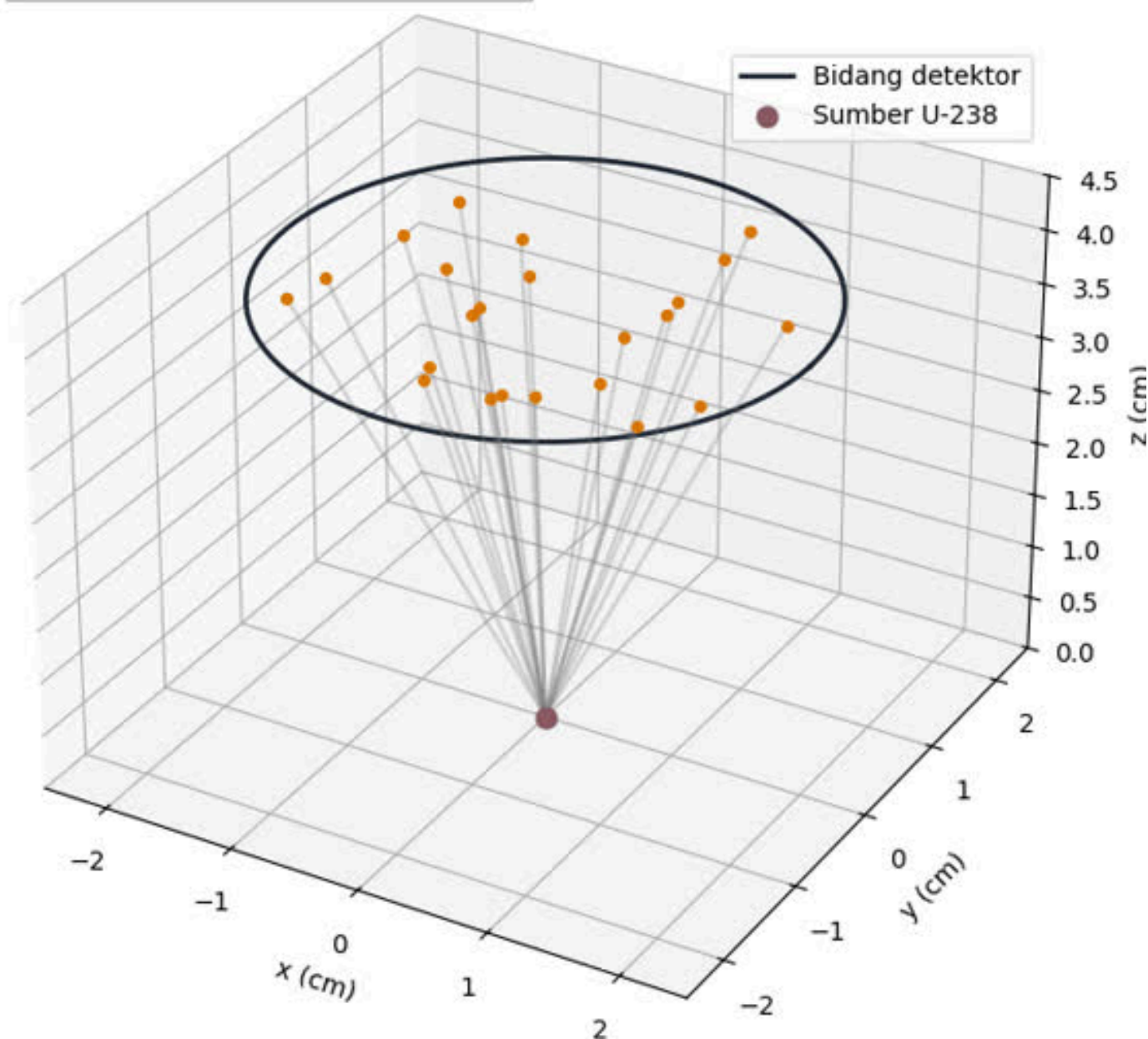
    if rank != 0:
        return None, rank, size
    return combine_batches(gathered), rank, size
```

- Setiap core akan diidentifikasi berdasarkan rank.
- Rank 0 bertindak sebagai Master (mengoordinasikan jalannya program, mengumpulkan data hasil, rendering visualisasi akhir).
- Komunikasi data menggunakan MPI.COMM_WORLD (antar cluster)

Hasil Visual

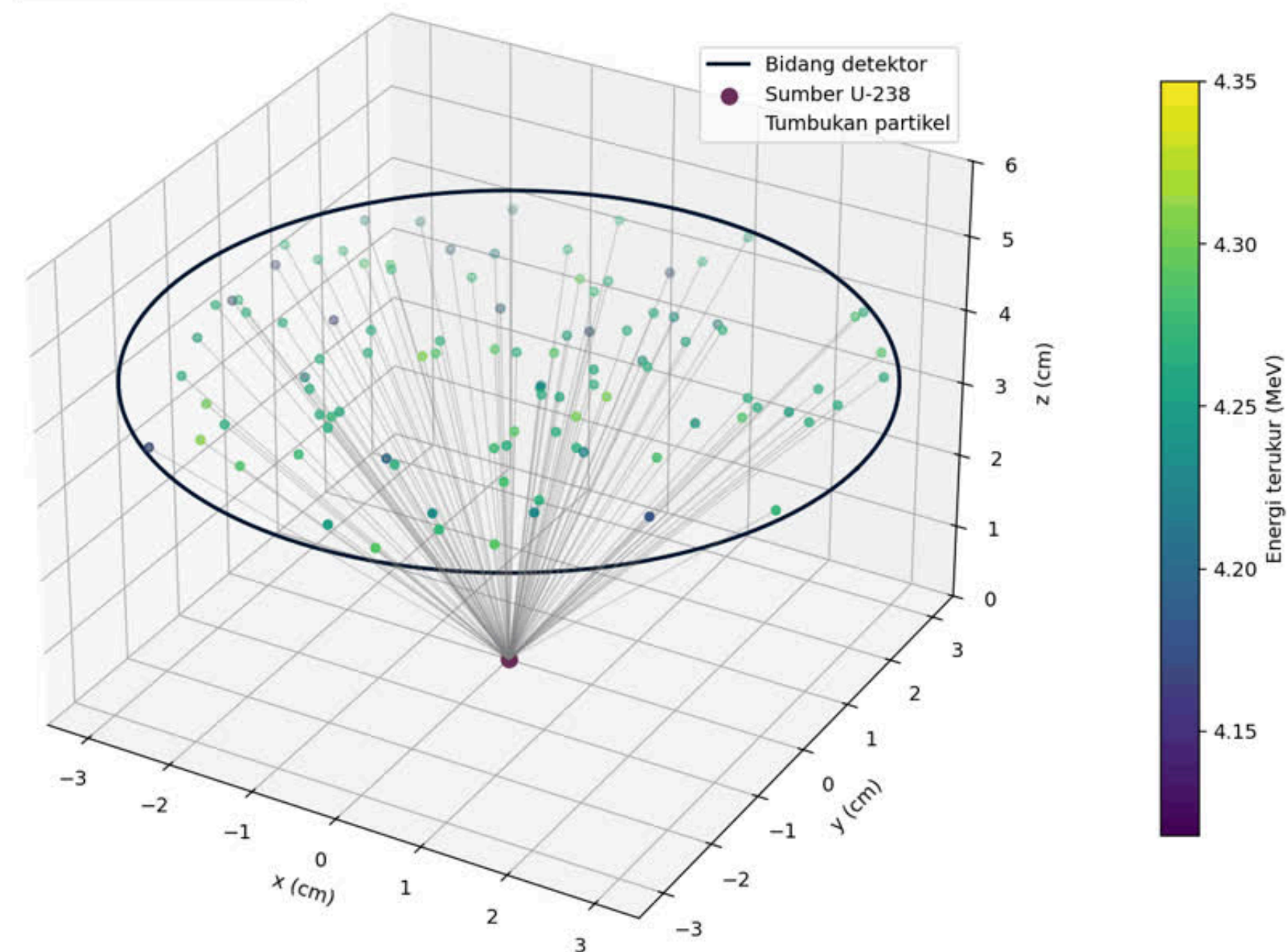
Akumulasi Deteksi 200 Partikel Alfa U-238

Deteksi Partikel: 23 / 200
Energi Terakhir = 4.278 MeV
Waktu Fisis Berjalan = 0.0243 detik



Akumulasi Deteksi 200 Partikel Alfa U-238 (MP | 8 worker | batch=20)

Deteksi: 100/200
Batch: 20 partikel/frame
E terakhir = 4.264 MeV
t fisis = 0.0170 s





Analisa Hasil

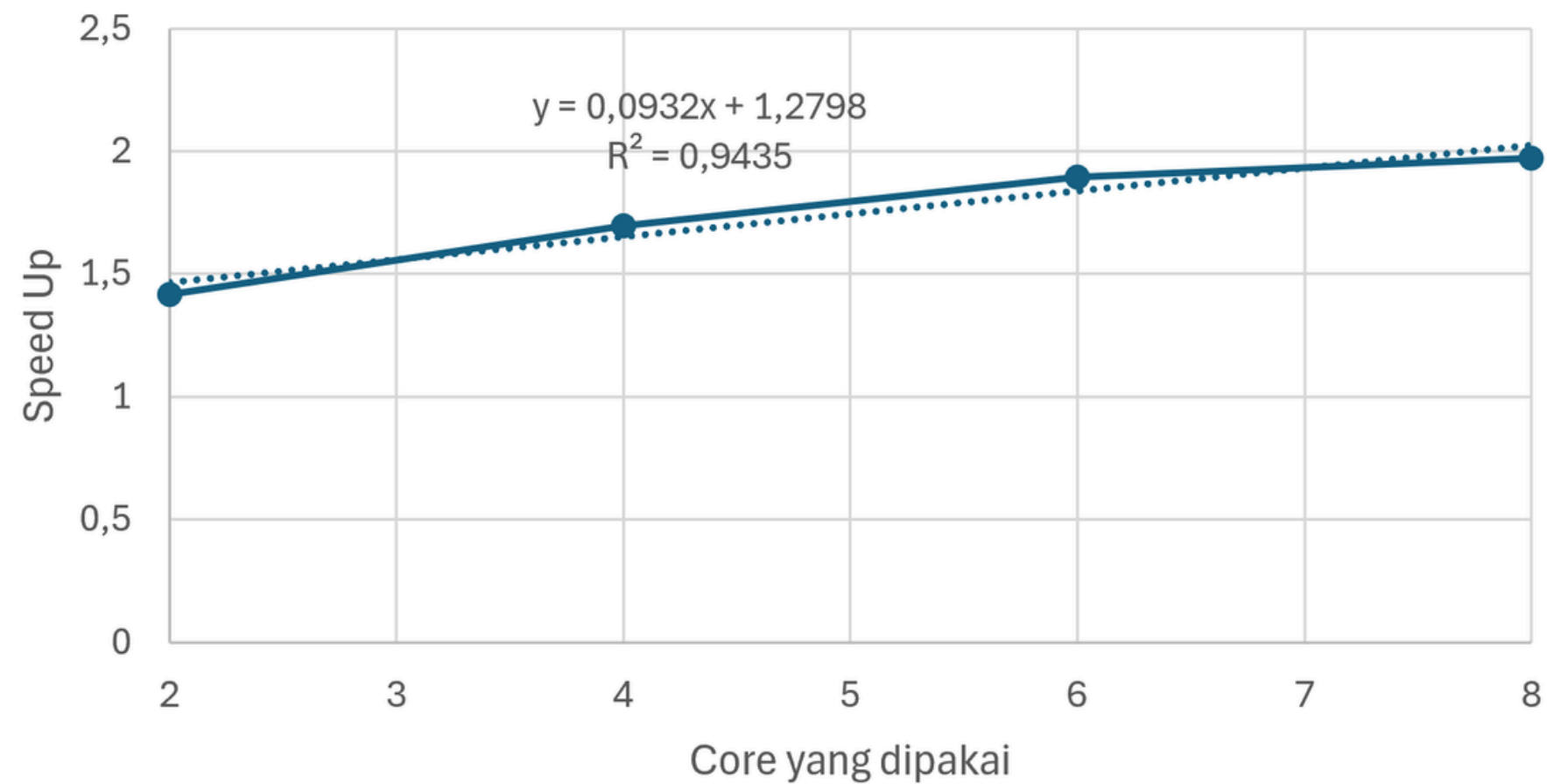
WAKTU SERIAL VISUALISASI (DETIK)
14,6

CORE	WAKTU VISUALISASI (DETIK)
2	10,3
4	8,6
6	7,7
8	7,4

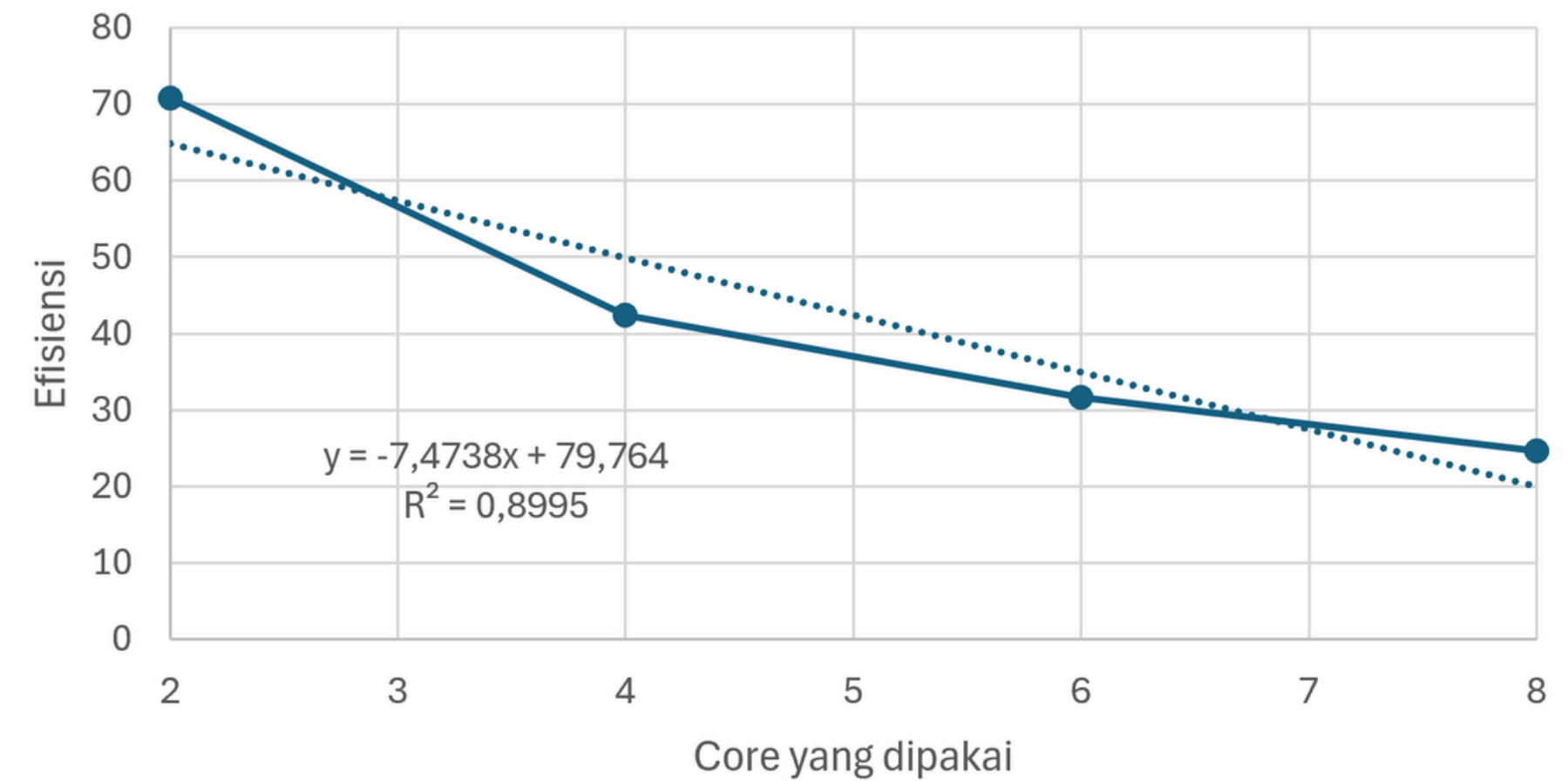
Speed up	
2	1,417475728
4	1,697674419
6	1,896103896
8	1,972972973
Efisiensi	
2	70,87378641
4	42,44186047
6	31,6017316
8	24,66216216

Analisa Hasil

Grafik Speed Up simulasi



Grafik Efisiensi Simulasi



Kendala dan Solusi

- Fisis

Peluruhan U-238 yang acak waktunya dan tidak linear.

Solusinya memakai pendekatan metode Monte Carlo dengan Inverse Transform Sampling.

Partikel alfa memancar ke segala arah, dan terdapat kemungkinan adanya partikel yang tidak terdeteksi di luar geometri sensor detektor.

Solusinya Menerapkan Sampling Geometri Kerucut (Solid Angle), membatasi sudut emisi agar partikel alfa hanya mengarah ke bidang cincin detektor, sehingga meningkatkan efisiensi kalkulasi data.

Kendala dan Solusi

- Komputasi

Kompleksitas Komputasi Gaya $O(N)^2$

Perhitungan gaya tolak-menolak Coulomb antar-partikel alfa memerlukan perhitungan interaksi setiap partikel, eksekusi serial sangat lambat saat jumlah data besar.

solusi berupa Strategi Data Parallelism (Multi-processing/MPI). Membagi partikel ke dalam beberapa kelompok batch independen

Penurunan Efisiensi pada Strong Scaling

saat data diuji (200 partikel) penambahan core justru menurunkan efisiensi, akibat besarnya Parallel Overhead

Solusi melalui pendekatan Weak Scaling (menaikkan jumlah data seiring penambahan core) dan memisahkan proses kalkulasi numerik dari rendering visualisasi Matplotlib.

Kesimpulan

- Penerapan komputasi paralel berhasil mempercepat waktu eksekusi secara keseluruhan.
- Speed Up menaik hampir linear tetapi melandai di core besar (*sub-linear speedup*), dan memvalidasi adanya hukum Amdahl.
- Efisiensi berbanding terbalik dengan menurun seiring bertambahnya core. Hal ini mencerminkan karakteristik Skalabilitas Kuat (Strong Scaling) yang rendah untuk ukuran problem yang kecil (200 partikel). akibat dominasi Parallel Overhead.



PRODI FISIKA
UNIVERSITAS BRAWIJAYA

TANYA

JAWAB

Copyright © 2025 Hanafi Hasan



PRODI FISIKA
UNIVERSITAS BRAWIJAYA

**TERIMA
KASIH**

Copyright © 2025 Hanafi Hasan